CÓDIGOCLARO · MATERIAL DE ESTUDIO

Python desde cero

Cuaderno DAM/DAW · Python 3.14 · ejemplos con biblioteca estándar, sin dependencias externas.

Intérprete y punto de entrada

Objetivo: Ejecutar un script desde consola sin que se imprima al importarlo desde otro módulo.

Python ejecuta las instrucciones en orden y delimita bloques mediante indentación. El guard de __main__ separa una demostración ejecutable de las definiciones reutilizables; al importar, el código dentro del guard no se ejecuta. Anota el intérprete usado y el directorio de trabajo para reproducir un problema.

Código · Python 3.14

```
def saludar(nombre):
    return f"Hola, {nombre}"

if __name__ == "__main__":
    print(saludar("Ada"))
```

Práctica: Importa el módulo desde principal.py. ¿Debe imprimirse el saludo automáticamente?

Respuesta razonada: No. Al importarse, __name__ contiene el nombre del módulo, no __main__. La función se puede reutilizar sin que el import tenga un efecto de impresión.

Tipos, conversiones y dinero

Objetivo: Convertir una entrada de texto y conservar un importe decimal exacto.

input devuelve str aunque la persona escriba números. Convierte de forma explícita y captura ValueError cuando la representación no sea válida. float usa coma flotante binaria; para un importe exacto, construye Decimal desde texto y define la regla de redondeo que usa la aplicación.

Código · Python 3.14

```
from decimal import Decimal
precio = Decimal("19.95")
unidades = int("3")
total = precio * unidades
print(total)  # 59.85
```

Práctica: ¿Por qué no conviene crear Decimal a partir de float 19.95?

Respuesta razonada: El float ya contiene una aproximación binaria al convertirse. Decimal desde la cadena conserva el importe decimal escrito; el redondeo debe definirse aparte según la regla contable.

Condiciones y validación

Objetivo: Clasificar una nota válida, aprobada o suspensa sin admitir valores fuera de escala.

if y elif seleccionan una rama. Valida primero el dominio y después evalúa el aprobado; de lo contrario, 12 podría aceptarse por ser mayor que cinco. Incluye casos frontera 0, 5 y 10 y separa entrada inválida de resultado suspenso.

Código · Python 3.14

```
def clasificar(nota):
    if not 0 <= nota <= 10:
        return "inválida"
    if nota >= 5:
        return "aprobada"
    return "suspensa"
```

Práctica: Calcula las respuestas para -1, 0, 5 y 10. ¿Qué ocurre con 12?

Respuesta razonada: Las salidas son inválida, suspensa, aprobada y aprobada; 12 también es inválida. Validar el rango antes del aprobado mantiene el contrato explícito.

Bucles y terminación

Objetivo: Sumar cantidades con un recorrido cuyo estado y final sean fáciles de verificar.

for recorre elementos; range genera enteros hasta un extremo que no incluye. El acumulador comienza en cero y cambia en cada iteración. Un while necesita una condición de salida y un avance demostrable. Prueba la colección vacía y el elemento final para detectar límites incorrectos.

Código · Python 3.14

```
pendientes = [2, 0, 3]
total = 0
for cantidad in pendientes:
    total += cantidad
print(total)  # 5
```

Práctica: Escribe el recorrido equivalente con índices. ¿Qué valores produce range(len(pendientes))?

Respuesta razonada: Produce 0, 1 y 2, uno por cada posición. El rango excluye el 3; añadir uno al límite intentaría acceder a un índice inexistente.

Listas, tuplas, conjuntos y diccionarios

Objetivo: Elegir una estructura para mantener orden y otra para evitar códigos duplicados.

list conserva secuencia mutable; tuple representa una secuencia que no se modifica en el sitio; set modela pertenencia sin duplicados; dict relaciona claves con valores. Copiar una lista anidada es superficial: los objetos internos pueden seguir compartidos. El orden de presentación debe fijarse explícitamente cuando sea requisito.

Código · Python 3.14

```
modulos = ["Programación", "Bases de datos"]
codigos_unicos = set(["PRG", "PRG", "BBDD"])
notas = {"PRG": 8, "BBDD": 7}
print(notas.get("PRG"))
```

Práctica: ¿Qué usarías para conservar notas en orden y qué para impedir códigos duplicados?

Respuesta razonada: Una lista conserva el orden de notas; un conjunto elimina duplicados de códigos. Si el orden de códigos también importa, usa dict como conjunto ordenado en Python moderno y documenta esa decisión.

Funciones y valores por defecto

Objetivo: Calcular una tasa de IVA y rechazar precios negativos con una función reutilizable.

Los parámetros nombrados aclaran la llamada. Los valores por defecto se evalúan al definir la función; evita una lista mutable por defecto cuando cada llamada deba recibir estado nuevo. Una función reutilizable devuelve valor y declara errores, en lugar de imprimir como efecto lateral. Para decimales comerciales, usa Decimal consistentemente.

Código · Python 3.14

```
from decimal import Decimal
def calcular_iva(precio, tipo=Decimal("0.21")):
    if precio < 0:
        raise ValueError("Precio negativo")
    return precio * (1 + tipo)
```

Práctica: ¿Qué excepción produce el precio -1 y qué devuelve 100 con tipo 0.10?

Respuesta razonada: El precio negativo lanza ValueError; 100 con tasa Decimal 0.10 devuelve 110.00. Mantén ambos operandos Decimal para no mezclar precisión decimal y binaria.

Módulos y entorno virtual

Objetivo: Aislar dependencias y comprobar qué intérprete ejecuta un proyecto de aula.

Un módulo es un fichero importable y un paquete agrupa módulos relacionados. El entorno virtual separa dependencias por proyecto; la biblioteca estándar no requiere pip. Usa el mismo intérprete para instalar y ejecutar paquetes, y registra versiones externas para que el equipo reproduzca el entorno.

Código · Python 3.14

```
python -m venv .venv
# PowerShell:
.\.venv\Scripts\Activate.ps1
python --version
python -m pip --version
```

Práctica: ¿Qué salida confirma que pip pertenece al entorno activo?

Respuesta razonada: La ruta mostrada por python -m pip debe apuntar a .venv. Usar pip mediante el propio intérprete reduce el riesgo de instalar en otro Python global.

Ficheros, JSON y errores

Objetivo: Leer un perfil UTF-8 y distinguir JSON válido de datos que cumplen el contrato.

with cierra el fichero tras leerlo. json.load analiza sintaxis, pero no comprueba el esquema de aplicación; valida tipo raíz y campos después. Distingue fichero ausente, JSON malformado y un documento válido con campos inválidos, porque la respuesta y el diagnóstico son distintos.

Código · Python 3.14 + json

```
import json
from pathlib import Path
with Path("alumno.json").open(encoding="utf-8") as f:
    datos = json.load(f)
if not isinstance(datos, dict) or not isinstance(datos.get("nombre"), str):
    raise ValueError("Perfil inválido")
```

Práctica: El fichero contiene []. ¿Por qué el parseo puede acabar bien y el perfil ser inválido?

Respuesta razonada: [] es JSON sintácticamente válido, pero la raíz no es un objeto con nombre. Comprueba el tipo antes de usar get y comunica el incumplimiento de esquema.

Clases y invariantes

Objetivo: Modelar una cuenta que rechace saldo inicial negativo y proteja su estado.

Una clase define comportamiento y cada instancia tiene su propio estado. __init__ recibe self y valida la invariante antes de guardar. Dataclass reduce código para datos, pero no añade automáticamente reglas de negocio. Expón operaciones válidas y evita ofrecer asignaciones que permitan romper el estado.

Código · Python 3.14

```
class Cuenta:
    def __init__(self, saldo):
        if saldo < 0:
            raise ValueError("Saldo negativo")
        self._saldo = saldo
    @property
    def saldo(self): return self._saldo
```

Práctica: Añade ingresar(cantidad). ¿Debe cambiar el saldo si cantidad es -5?

Respuesta razonada: No. Lanza ValueError antes de modificarlo. La cuenta mantiene el saldo anterior y conserva la misma regla para cada llamador.

Pruebas y depuración

Objetivo: Convertir el contrato de contar pares en pruebas repetibles para entradas límite.

Una prueba prepara una entrada, ejecuta una unidad y compara un resultado observable. Incluye caso habitual, vacío y valores negativos si el dominio los admite. Para depurar, reduce el fallo, observa valores intermedios y prueba una hipótesis cada vez; no ocultes la causa con un resultado inventado.

Código · unittest · Python 3.14

```
import unittest
def contar_pares(xs):
    return sum(1 for n in xs if n % 2 == 0)
class TestPares(unittest.TestCase):
    def test_vacio(self):
        self.assertEqual(contar_pares([]), 0)
```

Práctica: Añade pruebas para [2, 3, 4] y [-2, -1]. ¿Qué resultados esperas?

Respuesta razonada: Esperas 2 y 1. El resto módulo dos identifica pares negativos también; el caso vacío fija el valor neutro de la suma.

Fuentes oficiales

Documentación oficial Python 3.14 · Tutorial oficial Python 3.14 · Biblioteca estándar: json · Biblioteca estándar: pathlib · Biblioteca estándar: unittest